Functions in C Programming

Lecture 1 — Introduction

So today we are going to introduce you about what is function in C programming, so basically you have covered the basic topics of C like variables, data_types, conditional statements, loops, operators etc, now you are going to know about functions in C.

So you already know there exist a function from the beginning of your C programming journey, which you are writing in every of your source code and without that function your code will be not compiled and the compiler will throw you a error, so basically this function is called the "main" function which you are writing at the beginning inside which your statement or block of code is present, this function has a data_type 'int' or an integer and a return type value 'return 0' to make the compiler know that the program has been executed successfully.

[1.] What is a Function?

A function is a **block of code written outside your main function** to perform a specific task.

[2.] Function Declaration

General format:

```
function_data_type function_name(parameter1, parameter2, ...)
{
    // your block of code
    return statement;
}
```

[3.] Function Features

- **Reusability**: Make a block of code reusable — write once, call many times.
- **Modularity**: Breaks a large program into smaller, manageable chunks. Each function does one job — clean separation of logic.
- **Pass by value**: C passes arguments by value. The function gets a copy; the original variable is untouched.
- **Multiple parameters**: You can add multiple parameters along with their data types inside the function brackets **()**.
- **Local variables**: Variables declared inside a function are local — invisible outside it, mean you cannot call and use the variables or make the variables global and use everywhere in your source code, it will throw an error, i.e function variables are local.

[4.] What is a Function Prototype?

A prototype is like a **test model** — think of a car company making a blueprint before building the real car. Similarly, a function prototype tells the compiler about a function *before* it's actually

defined. It contains the return type, function name, and parameters. Always declare prototypes at the top of your code — it's good practice.

```
#include <stdio.h>

int area_ofrectangle(int l, int b);  // function prototype

int main()
{
    // your block of code
    return 0;
}

int area_ofrectangle(int l, int b)   // original function
{
    return l * b;
}
```

[5.] Parameters vs Arguments

A **parameter** is a variable (with its data type) declared inside the function's **()**. It stores the value when you call the function.

Example: `int area_ofrectangle(int l, int b)` — here **l** and **b** are parameters of type **int**.

An **argument** is the actual raw data (literal value) you pass when calling the function.

```
#include <stdio.h>

int area_ofrectangle(int l, int b);

int main()
{
    int area_rect = area_ofrectangle(15, 3);  // 15 and 3 are arguments
    printf("%d", area_rect);
    return 0;
}

int area_ofrectangle(int l, int b)
{
    return l * b;
}

// Output: 45
```

If you don't want to return anything, use the **void** data type:

```
#include <stdio.h>

void Greet(char name[]);
```

```c
int main()
{
    Greet("Soham");
    return 0;
}

void Greet(char name[])
{
    printf("Hello %s, how are you learning ?", name);
}

// Output: Hello Soham, how are you learning ?
```